UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Nombres Josue Yahir Vela Hernandez Osweld Omar Lozano Martínez Sylvia Yadira González Parra	**Matriculas** 2142881 2142366 2142962
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor:** Ing. Miguel Ángel Gómez Marroquin
Actividad: Diseño Arquitectural del sistema RESUMEN	**Fecha:** 17/10/2026

Contenido

1. Diagrama de clases *3*

2. Diagramas de secuencia *3*

2.1 Flujo normal **3**

2.2 Excepción **4**

3. Architecture Decision Records *4*

3.1 ADR 001 ..... **4**

3.2 ADR 002 ..... **5**

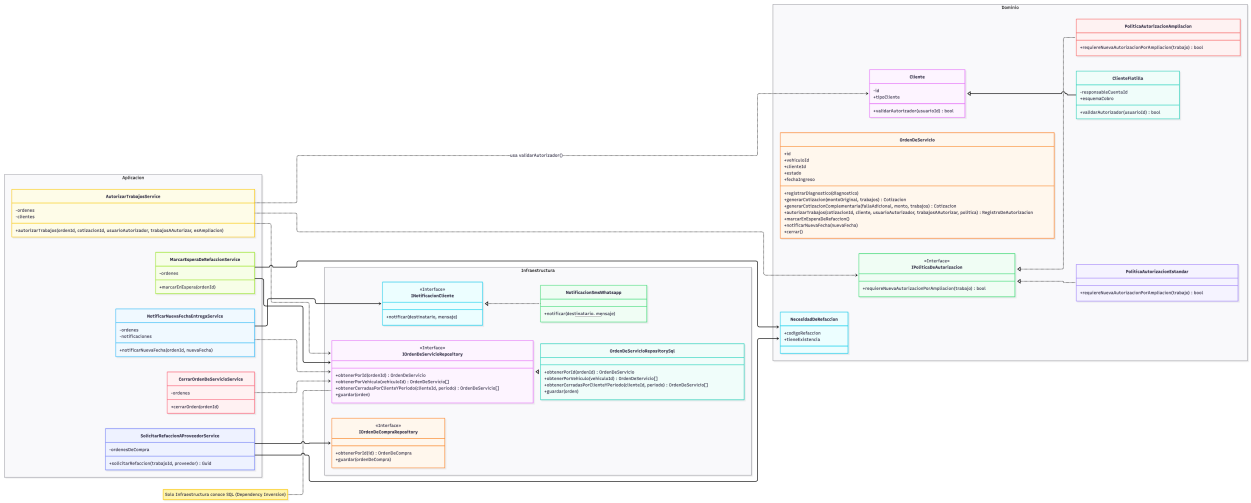
3.3 ADR 003 ..... **6**

3.4 ADR 004 ..... **6**

4. Trazabilidad *7*

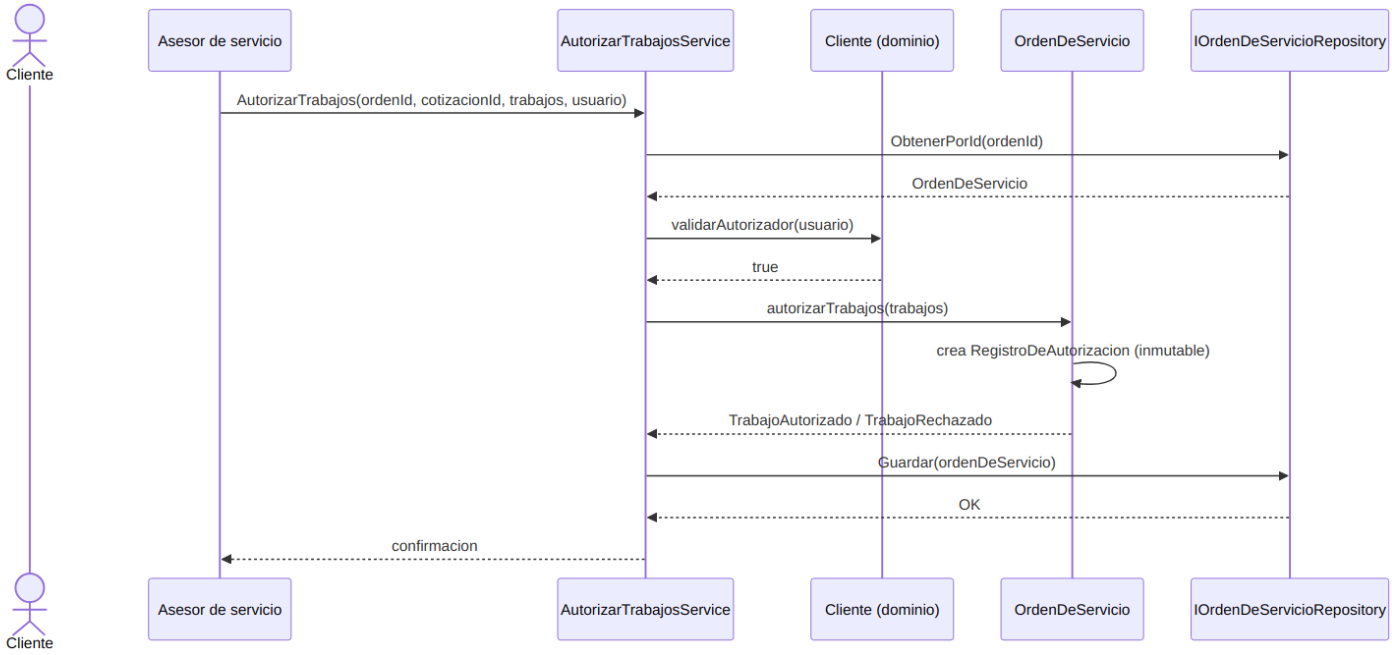
5. Bitácora de IA *9*

1. Diagrama de clases



2. Diagramas de secuencia

2.1 Flujo normal



2.2 Excepción



3. Architecture Decision Records

3.1 ADR 001

ADR-001	Contenido
Título	Selección del patrón de lógica de negocio
Estado	Aceptada
Contexto	`OrdenDeServicio` tiene 9 estados con transiciones condicionadas y más de 10 conceptos relacionados (Vehículo, Cliente, Diagnóstico, Cotización, Trabajo, Autorización, Refacción, Orden de Compra, Facturación Consolidada). El levantamiento documenta reglas transversales: umbral del 20% en ampliaciones (pregunta 12), escalamiento a 24h sin respuesta (pregunta 13), autorización parcial por trabajo (US-05/06) y trato diferenciado por tipo de cliente (BN-05). Se consideraron Transaction Script (rápido, pero repite cada regla en cada script), Table Module (resuelve consultas sobre conjuntos, pero no protege invariantes por instancia) y Domain Model (encapsula cada regla en el objeto que le corresponde)
Decisión	Domain Model como patrón principal para todo el flujo de servicio (OrdenDeServicio, Cotización, Trabajo, Autorización, Cliente/ClienteFlotilla). Table Module ligero, como excepción delimitada, únicamente para el módulo de reportes (US-15), donde no hay reglas de negocio que proteger, solo agregación de datos ya existentes
Consecuencias	Beneficio: las invariantes de estado y las reglas de autorización quedan protegidas en un solo lugar, sin repetirse en cada pantalla. Costo: mayor

	inversión de diseño inicial, porque hay que definir Aggregates, invariantes y Value Objects antes del primer caso de uso completo. Riesgo: el equipo debe evitar reducir las entidades a simples contenedores de datos. Trabajo posterior: documentar el mapa de dominio completo (Etapa 3) antes de construir la Service Layer
Trazabilidad	BN-06, US-03, US-04, US-08, pregunta 12 del levantamiento

3.2 ADR 002

ADR-002	Contenido
Título	Separar gestión de refacción sin existencia en dos servicios
Estado	Aceptada
Contexto	Un solo servicio (`GestionarRefaccionSinExistenciaService`) coordinaba en una misma operación dos Aggregates distintos: `OrdenDeCompra` (gestionado por Compras) y `OrdenDeServicio` (seguimiento del vehículo). Esto violaba SRP: un cambio en la política de compras obligaría a tocar una clase que también afecta el estado del vehículo frente al cliente, y viceversa (hallazgo #2, Etapa 7). Se consideró mantener un servicio único con transacción distribuida, frente a dos servicios independientes coordinados por evento
Decisión	Dos servicios de un solo Aggregate cada uno — `SolicitarRefaccionAProveedorService` y `MarcarEsperaDeRefaccionService`— coordinados por el Domain Event `RefaccionSinExistenciaDetectada`
Consecuencias	Beneficio: independencia real entre Compras y seguimiento del vehículo; cada Aggregate se modifica y persiste sin depender del otro. Costo: la consistencia entre “existe la orden de compra” y “el vehículo está en espera” pasa de ser inmediata (ACID) a eventual. Riesgo: si el mecanismo de eventos falla silenciosamente, un vehículo podría quedar “en espera” sin la orden de compra correspondiente. Trabajo posterior: definir una estrategia de reintento o reconciliación para el evento
Trazabilidad	BN-03, US-11, US-12, US-13, pregunta 9 del levantamiento

3.3 ADR 003

ADR-003	Contenido
Título	Autorización polimórfica en Cliente/ClienteFlotilla
Estado	Aceptada
Contexto	Existía un servicio aparte, `AutorizarComoResponsableDeFlotillaService`, que duplicaba casi por completo la lógica de `AutorizarTrabajosService` solo para cambiar quién podía autorizar. Esto violaba Liskov: `ClienteFlotilla` no actuaba como un verdadero sustituto de `Cliente`, sino como una ruta de código distinta (hallazgo #4, Etapa 7). Se consideró mantener servicios separados por tipo de cliente, frente a un solo servicio con un método polimórfico
Decisión	Un solo servicio (`AutorizarTrabajosService`) que delega "quién puede autorizar" a un método polimórfico `validarAutorizador(usuario)`, implementado de forma distinta en `Cliente` (valida al titular) y en `ClienteFlotilla` (valida al responsable de cuenta)
Consecuencias	Beneficio: un solo punto de entrada para "autorizar trabajos" en los tres casos de uso que lo necesitan (CU-01, CU-02, CU-05). Costo: ninguno significativo. Riesgo: si aparece un tercer tipo de cliente con reglas muy distintas, habrá que confirmar que el polimorfismo simple siga siendo suficiente. Trabajo posterior: ninguno pendiente para el alcance actual
Trazabilidad	BN-05, US-16, pregunta 8 del levantamiento

3.4 ADR 004

ADR-004	Contenido
Título	Política intercambiable para el umbral de autorización (20%)
Estado	Aceptada
Contexto	`ServicioDeAutorizacion` estaba planteado como una sola clase que acumularía un condicional por cada regla nueva de autorización. Esto violaba Open/Closed: cada regla nueva obligaría a modificar un método ya existente en vez de extender el sistema (hallazgo #3, Etapa 7). Se consideró un único método con condicionales crecientes, frente a una interfaz con una implementación por regla

Decisión	Interfaz `IPoliticaDeAutorizacion` con políticas intercambiables (`PoliticaAutorizacionEstandar`, `PoliticaAutorizacionAmpliacion`), seleccionada según el caso en vez de ramificada internamente
Consecuencias	Beneficio: una regla nueva se agrega como una clase nueva, sin modificar las políticas existentes. Costo: más clases pequeñas, lo que exige una convención clara de nombres. Riesgo: ninguno relevante detectado. Trabajo posterior: queda pendiente con el stakeholder si el umbral del 20% debe aplicar también a fallas adicionales pequeñas, no solo a ampliaciones sobre trabajos ya autorizados
Trazabilidad	Pregunta 12 del levantamiento; hallazgo 6 de la Etapa 3

4. Trazabilidad

ID Necesidad (BN)	Descripción de la Necesidad	Historias de Usuario (US) asociadas	Criterios de Éxito Impactados
BN-01	Reducir disputas por trabajos autorizados mediante evidencia digital.	US-02 (Fotos ingreso) US-07 (Registro digital de autorización) US-20 (Fotos entrega)	• Reducción medible de disputas en 3 meses. • 100% de autorizaciones con evidencia digital.
BN-02	Aumentar visibilidad del estado del vehículo en tiempo real.	US-14 (Consulta de estado) US-15 (Reportes gerente)	• Mejora medible de satisfacción en encuestas.
BN-03	Comunicar oportunamente retrasos por falta de refacciones.	US-11 (Orden compra) US-12 (Estado espera refacción)	• Tiempo promedio de 2 horas de comunicación de retrasos.

		US-13 (Notificar nueva fecha)	
BN-04	Registrar fallas adicionales y su autorización.	US-09 (Reporte falla adicional) US-10 (Cotización complementaria)	• 100% de autorizaciones con evidencia digital.
BN-05	Gestión de flotillas (autorización y facturación diferenciada).	US-16 (Autorización delegada) US-17 (Facturación mensual) US-18 (Prioridad contractual)	• Mejora medible de satisfacción del cliente (Flotillas).
BN-06	Estandarizar proceso de cotización total o parcial.	US-01, US-03, US-04(Flujo inicial) US-05, US-06(Aprobación parcial) US-08 (Visibilidad técnico)	• Reducción medible de disputas en 3 meses.

5. Bitácora de IA

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	Analizar continuidad con la Unidad 2 antes de diseñar	Comencemos con la etapa número 1: ¿qué hallazgos encuentras en el documento de la Unidad 2 que sean importantes, hay correcciones que se deban realizar?	La IA identificó 5 hallazgos: carril del Gerente del taller ausente en el BPMN; brecha de política para vehículo no recogido (pregunta 14) sin US asociada; restricción de conexión inestable sin decisión de persistencia; contradicción entre pregunta 10 y pregunta 12 del levantamiento, mezcladas en US-10; y Autorización sin tipo de elemento de dominio definido.	La contradicción de US-10 (falla adicional vs. umbral del 20%) no estaba resuelta por el equipo en la Unidad 2; se registró como ambigüedad real, no se inventó una regla nueva para resolverla.	El equipo confirmó los 5 hallazgos y decidió avanzar con ellos declarados como supuestos pendientes.
2	Seleccionar y verificar los casos de uso de la Etapa 1	Por favor mantén la tabla con el formato ID, Caso de uso, Actor, Reglas relacionadas, Resultado esperado; verifica que los casos aparecen en las user stories o en el BPMN, al menos en uno de los dos	La IA propuso 5 casos de uso (CU-01 a CU-05) con su actor, regla y resultado, y hasta ese punto solo había comparado contra la captura de pantalla del BPMN, no contra el archivo fuente.	La IA reportó una discrepancia de actor entre el BPMN y las user stories (US-10 y US-17) basada en una lectura visual de la captura, que resultó ser un error de interpretación de la IA, no un error real del documento.	El equipo aportó los archivos fuente .drawio del BPMN y del UML para que la IA verificara contra el XML real en vez de la imagen; al revisar el archivo correcto, la IA retiró el hallazgo de discrepancia. El equipo también decidió agregar CU-05 (BN-05, sin cobertura en la propuesta inicial de 4

					casos) y descartó un BPMN y un UML alternos (archivos .drawio adicionales) que no correspondían a la versión final de la Unidad 2.
3	Comparar patrones de lógica de negocio (Etapa 2)	Continuemos con la Etapa 2 donde vas a proporcionarnos la tabla donde se elege el patrón de la lógica de negocio, el formato debe mantenerse tal cual al documento y vas a agregar evidencia de la U2	La IA entregó la tabla comparativa (Transaction Script, Table Module, Domain Model) con 5 criterios y evidencia de la Unidad 2 en cada fila, y propuso Domain Model como patrón principal con Table Module ligero para reportes (US-15).	La actividad exige una decisión argumentada de 1 a 2 páginas; en la primera entrega solo se incluyó una oración de decisión, sin la justificación extendida.	El equipo aprobó la decisión (Domain Model) sin pedir cambios al razonamiento, y en una revisión posterior se agrego la justificación extendida de una página antes de la entrega final.
4	Construir el mapa de dominio (Etapa 3)	Ahora vamos a la etapa tres, dame un bosquejo del diagrama de dominio, respetando el los archivos que ya tienes, el diagrama debe de entregarse en codigo para mermaid, asi como tambien los demas requisitos que pide esta etapa (Entities, value objects, domain services, etc)	La IA entregó el diagrama Mermaid con Aggregates, Entities, Value Objects, Domain Services y Domain Events; el glosario actualizado; la ficha de cada Aggregate (raíz, responsabilidades, invariantes, eventos); y 6 decisiones dudosas o descartadas, incluyendo el umbral del 20% sin confirmar con el stakeholder.	Antes de este mapa, el equipo erroneamente subió un UML alterno (diagrama_uml_ed.drawio) que la IA detectó como inconsistente: le faltaban Trabajo, Autorización, ClienteFlotilla y FacturaciónConsolidada, y no distinguía tipo de cliente.	El equipo revisó la comparación que hizo la IA entre ambos UML y decidió descartar el archivo alterno, confirmando que el mapa de dominio construido con el UML del PDF original de la unidad 2 era el correcto.

5	Diseñar la Service Layer (Etapa 4)	Continuemos con la etapa 4, donde vas a diseñar la service layer respetando el formato de la tabla, debes de verificar obligatoriamente que cumpla con las 5 preguntas de revision para asegurar el cumplimiento de los requisitos de esta etapa	La IA entregó 9 Application Services con entrada, colaboradores, salida y reglas que delegan, y respondió explícitamente las 5 preguntas de revisión obligatoria (nombre de negocio, coordinación vs. decisión, dependencia de interfaces, límite transaccional, errores en términos de negocio).	—	El equipo aprobó la tabla sin cambios en este punto; sin embargo, las correcciones a esta etapa llegaron después, en la iteración 7.
6	Separar persistencia y transacciones (Etapa 5)	En esta etapa, nos proporcionaras las de interfaces de repository en terminos de dominio sin adentrarte en lenguaje SQL, Tambien, realizaras el bosquejo de la tabla de unit of work respetando el formato que solicita la actividad	La IA entregó las interfaces de Repository en términos del dominio (sin SQL) y la tabla de Unit of Work explicando dónde inicia cada transacción, qué incluye y qué pasa ante un error.	Quedó sin resolver si el Unit of Work debe contemplar la conexión inestable de las tablets mencionada en el BRD; se declaró como hallazgo pendiente y no se inventó una estrategia de reintento.	El equipo aceptó la propuesta y decidió dejar pendiente de validar la estrategia de conexión inestable.
7	Separar Commands y Queries (Etapa 6)	En esta etapa nos apoyaras proponiendo 3 commands y 3 queries separados, deben de contentes handler, entrada, resultado y justificacion basandote en el trabajo que ya existe de etapas anteriores, deberas cumplir los demas requisitos que solicita la actividad y justificar por que algunos CU no	La IA propuso 3 Commands y 3 Queries con handler, datos de entrada, resultado y justificación, con DTOs de lectura independientes del modelo de escritura, y justificó por qué CU-05 y el registro de ingreso no necesitan esta separación.	—	El equipo aprobó la propuesta sin cambios en este punto.

		requieren esa separacion			
8	Revisar la propuesta con SOLID y aplicar correcciones (Etapa 7)	Sí (continuar a la Etapa 7) / después: ¿las correcciones dónde se deben hacer? / rediseña Etapa 3, 4 y 5 con las correcciones	La IA entregó 6 hallazgos (2 confirmaciones: S y D; 2 correcciones: S y L; 2 riesgos: O e I) con evidencia e impacto, y luego mapeó cada corrección a los documentos exactos que debían modificarse.	Las correcciones de SRP (dividir el servicio de refacciones), OCP (política de autorización intercambiable) y LSP (autorización polimórfica de flotillas) no estaban aplicadas en la primera versión de las Etapas 3 a 5.	El equipo pregunto a la IA explícitamente dónde debía aplicarse cada corrección antes de aceptarlas, y después solicitó el rediseño de las Etapas 3, 4 y 5 con las 3 correcciones ya incorporadas.
9	Representar la arquitectura: diagramas y ADR (Etapa 8)	entregame el bosquejo en codigo de mermaid para el diagrama de clases separado en capas mencionadas en el documento, tambien el codigo para un diagrama de secuencia en flujo normal y otro para un excepcion, asi como tambien 4 ADR	La IA entregó el diagrama de clases por capas, dos diagramas de secuencia (flujo normal y excepción) y 4 ADR con los 7 campos exigidos (Título, Estado, Contexto, Alternativas, Decisión, Consecuencias, Trazabilidad).	El diagrama de clases inicial no incluía IOrdenDeCompraRepository ni NecesidadDeRefaccion, pese a que el diagrama de secuencia de excepción (CU-03) sí los usa como participantes, lo que violaba la consistencia exigida entre ambos diagramas.	El equipo pidió verificar el cumplimiento contra la rúbrica real de la actividad; tras detectarse la inconsistencia, se corrigió el diagrama de clases para incluir ambos elementos.

10	Validar la propuesta y preparar los entregables formales (Etapa 9 y entregables)	Valida los requisitos del documento de la U3 vs la informacion que me has entregado, si todo esta correcto entregame los documentos para yo poderlos editar manualmente y un codigo sencillo base en algun lenguaje de programacion facil de desplegar	La IA entregó la validación de las 7 preguntas de la Etapa 9 y generó el Informe de Arquitectura (Word), la Matriz de Trazabilidad (Excel) y un código base en C#, con los diagramas renderizados en PNG.	-	El equipo solicitó verificar cada etapa contra la rúbrica real de la actividad, corregir lo que no cumpliera, crear esta bitácora con el formato correcto para rellenar, y rehacer el código base en React con JavaScript en su versión más reciente en lugar de C#, ya que se decidio desplegar el proyecto como web para facilitar su uso

6. Link de Github

https://github.com/yahirvela19/Equipo-6-Dise-o-arquitectural-AUTOFIXPRO/tree/main/src